

The endpoints were tested using FastAPI Swagger UI.

GET /customers/count

GET

/api/v1/orders/count

Orders Count

⌵

Get total number of orders

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:65270/api/v1/orders/count' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:65270/api/v1/orders/count
```

Server response

Code	Details
200	Response body <pre>{ "orders": 326 }</pre> Download Response headers <pre>content-length: 14 content-type: application/json date: Sun, 10 May 2026 15:27:58 GMT server: uvicorn</pre>

Responses

GET /orderdetails/count

GET

/api/v1/orderdetails/count

Orderdetails Count

⌵

Get total number of order details

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:65270/api/v1/orderdetails/count' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:65270/api/v1/orderdetails/count
```

Server response

Code	Details
200	Response body <pre>{ "orderdetails": 2996 }</pre> Download Response headers <pre>content-length: 21 content-type: application/json date: Sun, 10 May 2026 15:29:03 GMT server: uvicorn</pre>

Responses

GET /overall_counts response

GET

/api/v1/overall_counts

Overall Counts

⌵

Get counts from all database tables concurrently

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:65270/api/v1/overall_counts' \
  -H 'accept: application/json'
```

⌵

Request URL

http://127.0.0.1:65270/api/v1/overall_counts

Server response

Code

Details

200

Response body

```
{
  "customers": 122,
  "orders": 326,
  "products": 110,
  "employees": 23,
  "offices": 7,
  "payments": 273,
  "orderdetails": 2096,
  "productlines": 7,
  "execution_time_seconds": 0.0107
}
```

⌵

Download

Response headers

```
content-length: 156
content-type: application/json
date: Sun, 10 May 2026 15:30:00 GMT
server: uvicorn
```

Logs in app.log

```
app.log
1 2026-05-10 20:31:19,792 - INFO - Root endpoint accessed
2 2026-05-10 20:31:37,109 - INFO - Root endpoint accessed
3 2026-05-10 20:31:41,983 - INFO - GET request received for /customers with skip=0, limit=100
4 2026-05-10 20:31:41,983 - INFO - Fetching customers with skip=0, limit=100
5 2026-05-10 20:31:41,993 - INFO - Retrieved 100 customers
6 2026-05-10 20:31:41,993 - INFO - Returning 100 customers
7 2026-05-10 20:31:45,335 - INFO - POST request received for /customers with data: Test Customer
8 2026-05-10 20:31:45,335 - INFO - Creating new customer: Test Customer
9 2026-05-10 20:32:55,401 - INFO - POST request received for /customers with data: Test Customer
10 2026-05-10 20:32:55,401 - INFO - Creating new customer: Test Customer
11 2026-05-10 20:32:55,418 - INFO - Successfully created customer with ID 497
12 2026-05-10 20:32:55,418 - INFO - Created new customer with ID 497
13 2026-05-10 20:33:30,592 - INFO - GET request received for /customers/497
14 2026-05-10 20:33:30,593 - INFO - Fetching customer ID 497
15 2026-05-10 20:33:30,595 - INFO - Successfully retrieved customer ID 497
16 2026-05-10 20:33:30,596 - INFO - Returning customer ID 497
17 2026-05-10 20:33:37,856 - INFO - PUT request received for /customers/497
18 2026-05-10 20:33:37,856 - INFO - Updating customer ID 497
19 2026-05-10 20:33:37,865 - INFO - Successfully updated customer ID 497
20 2026-05-10 20:33:37,865 - INFO - Updated customer ID 497
21 2026-05-10 20:33:54,058 - INFO - GET request received for /customers with skip=0, limit=5
22 2026-05-10 20:33:54,058 - INFO - Fetching customers with skip=0, limit=5
23 2026-05-10 20:33:54,061 - INFO - Retrieved 5 customers
24 2026-05-10 20:33:54,061 - INFO - Returning 5 customers
25 2026-05-10 20:34:09,154 - INFO - DELETE request received for /customers/497
26 2026-05-10 20:34:09,154 - INFO - Deleting customer ID 497
27 2026-05-10 20:34:09,163 - INFO - Successfully deleted customer ID 497
28 2026-05-10 20:34:09,163 - INFO - Deleted customer ID 497
29 2026-05-10 20:34:34,422 - INFO - Root endpoint accessed
30 2026-05-10 20:34:34,995 - INFO - Root endpoint accessed
31 2026-05-10 20:41:17,839 - INFO - GET request received for /customers with skip=0, limit=100
32 2026-05-10 20:41:17,840 - INFO - Fetching customers with skip=0, limit=100
33 2026-05-10 20:41:17,844 - INFO - Retrieved 100 customers
```

Conclusion

In this task, a concurrent dashboard API was successfully developed using FastAPI, SQLAlchemy, and Python asyncio. Separate modular endpoints were implemented for each database table, followed by an aggregated dashboard endpoint capable of executing all database queries concurrently.

The use of `asyncio.gather()` significantly improved performance by avoiding sequential delays. Centralized logging improved observability and debugging, while modular architecture ensured maintainability and scalability.

This task demonstrated practical backend engineering concepts including concurrency, asynchronous programming, modular API design, logging, and scalable system architecture.